

Substitution in Arrays

- We haven't discussed how to understand array assignments yet. If we assign an expression to $b[e]$, the index e can only be evaluated at runtime; which can lead to the following problem.

Assuming $b[j]$ and $b[k]$ are safe and $j \neq k$, then what is $wp(b[j] := b[j] + 1, b[k] < b[j])$?

- Some might say it is $b[k] < b[j] + 1$ but what if k and j evaluated to the same integer at runtime? Will we only substitute $b[j]$ with $b[j] + 1$? Or will we substitute both $b[j]$ and $b[k]$ with $b[j] + 1$ (and if so, we will get the weakest precondition being $b[j] + 1 < b[j] + 1 \Leftrightarrow F$)?

- We need to figure out how to understand substitutions in arrays $(e)[e_1 / b[e_0]]$ first.
- How to understand $(b[m])[6 / d[2]]$ where m is a variable or named constant?
 - If b and d are different arrays ($b \neq d$), then this is simple: there will be no expression $d[2]$ in array $b[m]$ then $(b[m])[6 / d[2]] \equiv b[m]$.
 - If b and d are the same array ($b \equiv d$), then we need to consider whether $m = 2$: if $m = 2$ then $(b[m])[6 / d[2]] \equiv 6$ or else it $\equiv b[m]$.

$b[d[2] + 1]$

- How to understand $(b[e])[6 / d[2]]$ where e is an expression?
 - If b and d are different arrays, then $b[e] \neq d[2]$ and we need to look recursively into e since expression $d[2]$ might appear in e ; then, $(b[e])[6 / d[2]] \equiv b[e[6 / d[2]]]$.
 - If b and d are the same array, we need to evaluate e . If e is evaluated to 2 at run time, then $(b[e])[6 / d[2]] \equiv 6$, or else we will look recursively into e like in the above case.

- Here we give the definition of syntactic substitution in arrays:

$$(b[e_2])[e_1 / b[e_0]] \equiv \text{if } e_2 = e_0 \text{ then } e_1 \text{ else } b[e_2] \text{ fi, where } e_2' \equiv (e_2)[e_1 / b[e_0]].$$

- This definition covers all cases in Example 1 and 2 while the substitution happens in the same array:
 - When e_2 is a named constant, aka $e_2 \equiv k$, then we get $e_2' \equiv k[e_1 / b[e_0]] \equiv k$, and then $(b[k])[e_1 / b[e_0]] \equiv \text{if } k = e_0 \text{ then } e_1 \text{ else } b[k] \text{ fi}$.

- Finish the following syntactic substitutions.

- $(b[k])[5 / b[0]] \equiv \text{if } k = 0 \text{ then } 5 \text{ else } b[k] \text{ fi}$
- $(b[k])[e_0 / b[j]] \equiv \text{if } k = j \text{ then } e_0 \text{ else } b[k] \text{ fi}$
- $(b[k])[b[j] + 1 / b[j]] \equiv \text{if } k = j \text{ then } b[j] + 1 \text{ else } b[k] \text{ fi}$

Note that, we will keep e_1 (in this case $b[j] + 1$) as it is, even if it involves b .

- $(b[k])[b[j] / b[b[k]]] \equiv \text{if } k = b[k] \text{ then } b[j] \text{ else } b[k] \text{ fi}$

- $(b[b[k]])[5 / b[0]]$

The inner $b[k]$ need to be taken care first, and $(b[k])[5 / b[0]] \equiv \text{if } k = 0 \text{ then } 5 \text{ else } b[k] \text{ fi}$. Then,

$$(b[b[k]])[5 / b[0]]$$

$$\equiv \text{if } (\text{if } k = 0 \text{ then } 5 \text{ else } b[k] \text{ fi}) = 0 \text{ then } 5 \text{ else } b[\text{if } k = 0 \text{ then } 5 \text{ else } b[k] \text{ fi}] \text{ fi}$$

$$\rightarrow \text{if } k = 0 \text{ then } b[5] \text{ else if } b[k] = 0 \text{ then } 5 \text{ else } b[b[k]] \text{ fi fi}$$



- In Example 4.e) we “logically simplified” a complicated expression to a “shorter” expression. Formally, we call this operation **optimization**, it means we replace an expression/predicate with a “shorter” or “simpler” expression/predicate that is semantically equal. It is written as $e_1 \mapsto e_2$ (“ e_1 optimizes to e_2 ”).
 - We introduce this definition here because syntactic substitutions in arrays usually end up with a long and complicated text (either expression or predicate). It can be useful to shorten it first before execution, similarly to how compilers can optimize code.
 - The optimization is done in a static way: the optimization is done before the code runs.
 - Since to $e_1 \mapsto e_2$ we need $e_1 \Leftrightarrow e_2$, it is okay to just use “ $\Leftrightarrow$ ” to represent optimization. But “ $e_1 \Leftrightarrow e_2$ ” emphasizes that e_1 and e_2 are semantically equal, and “ $e_1 \mapsto e_2$ ” emphasizes that e_1 is (or can be) optimized to e_2 .

Rules for Optimizing Condition Expressions

Let’s identify some general rules for optimizing conditional expressions and predicates involving them. This will let us simplify calculation of wlp or wp for array assignments.

- **(if T then e_1 else e_2 fi) $\mapsto e_1$**
- **(if F then e_1 else e_2 fi) $\mapsto e_2$**
- **(if B then e else e fi) $\mapsto e$**
- If we know that $B \Rightarrow e_1 = e_2$, then **(if B then e_1 else e_2 fi) $\mapsto e_2$** .
 - Since B can imply that $e_1 = e_2$, then no matter whether B is true or not, we always have e_2 .
- If we know that $\neg B \Rightarrow e_1 = e_2$, then **(if B then e_1 else e_2 fi) $\mapsto e_1$** .

Let op_1 be a unary operator, such as “ $\neg$ ”...; and op_2 be a binary operator such as “ $+$ ”, “ $<$ ” ...

- **op_1 (if B then e_1 else e_2 fi) $\mapsto$ if B then $op_1(e_1)$ else $op_1(e_2)$ fi**
- **(if B then e_1 else e_2 fi) $op_2 e_3 \mapsto$ if B then $e_1 op_2 e_3$ else $e_2 op_2 e_3$ fi**
- **b [if B then e_1 else e_2 fi] $\mapsto$ if B then $b[e_1]$ else $b[e_2]$ fi**
- **f (if B then e_1 else e_2 fi) $\mapsto$ if B then $f(e_1)$ else $f(e_2)$ fi**

Let B, B_1, B_2 be Boolean expression.

- **(if B then B_1 else B_2 fi) $\mapsto (B \wedge B_1) \vee (\neg B \wedge B_2)$**
- **(if B then B_1 else B_2 fi) $\mapsto (B \rightarrow B_1) \wedge (\neg B \rightarrow B_2)$**
- **(if B then B_1 else F fi) $\mapsto (B \wedge B_1)$**
 - **(if B then B_1 else F fi) $\Leftrightarrow (B \wedge B_1) \vee (\neg B \wedge F) \Leftrightarrow (B \wedge B_1)$.**
- **(if B then F else B_2 fi) $\mapsto (\neg B \wedge B_2)$**
- **(if B then B_1 else T fi) $\mapsto (B \rightarrow B_1)$**
- **(if B then B_1 else T fi) $\mapsto (\neg B \vee B_1)$**
 - **(if B then B_1 else T fi) $\Leftrightarrow (B \rightarrow B_1) \wedge (\neg B \rightarrow T) \Leftrightarrow (B \rightarrow B_1)$**
- **(if B then T else B_2 fi) $\mapsto (\neg B \rightarrow B_2)$**
- **(if B then T else B_2 fi) $\mapsto (B \vee B_2)$**

Now, let’s go back to the first question of this chapter.

4. Let j and k be two named constants that are at least 0 and less than $size(b)$. Calculate $wp(b[j] := b[j] + 1, b[k] < b[j])$, assuming $b[j]$ and $b[k]$ are safe.
 $wp(b[j] := b[j] + 1, b[k] < b[j])$

$$\begin{aligned}
&\equiv (b[k] < b[j]) [b[j] + 1 / b[j]] \\
&\equiv (b[k]) [b[j] + 1 / b[j]] < (b[j]) [b[j] + 1 / b[j]] \\
&\equiv (b[k]) [b[j] + 1 / b[j]] < \text{if } j = j \text{ then } b[j] + 1 \text{ else } b[j] \text{ fi} \\
&\mapsto (\text{if } k = j \text{ then } b[j] + 1 \text{ else } b[k] \text{ fi}) < (b[j] + 1) \\
&\mapsto \text{if } k = j \text{ then } (b[j] + 1) < (b[j] + 1) \text{ else } b[k] < (b[j] + 1) \text{ fi} \\
&\mapsto \text{if } k = j \text{ then } F \text{ else } b[k] < (b[j] + 1) \text{ fi} \\
&\mapsto k \neq j \wedge b[k] < (b[j] + 1)
\end{aligned}$$

- This gives us a valid triple $\{k \neq j \wedge b[k] < (b[j] + 1)\} b[j] := b[j] + 1 \{b[k] < b[j]\}$

- Correct a full proof outline of a program that swaps the values of primitive-type variables x and y .
 - To swap the values of x and y , we need the help of a temporary variable u , then we can create the following minimal proof outline:

$$\{x = x_0 \wedge y = y_0\} u := x; x := y; y := u \{x = y_0 \wedge y = x_0\}$$

- We can keep using backward assignments to create the following full proof outline:
 $\{x = x_0 \wedge y = y_0\} u := x; \{y = y_0 \wedge u = x_0\} x := y; \{x = y_0 \wedge u = x_0\} y := u \{x = y_0 \wedge y = x_0\}$

- Create a full proof outline of a program that swaps $b[m]$ and $b[n]$, assuming that m and n are natural numbers less than $\text{size}(b)$.

- Like question 6, we need to prove the following minimal proof outline:

$$\{b[m] = c \wedge b[n] = d\} u := b[m]; b[m] := b[n]; b[n] := u \{b[m] = d \wedge b[n] = c\}$$

- If we keep using backward assignments, then we can come up with the following full proof outline:
 $\{b[m] = c \wedge b[n] = d\} \{q_0\} u := b[m]; \{q_1\} b[m] := b[n]; \{q_2\} b[n] := u \{b[m] = d \wedge b[n] = c\}$

Let's calculate q_2, q_1 and q_0 . Note that, we also need to prove $b[m] = c \wedge b[n] = d \Rightarrow q_0$.

$$\begin{aligned}
q_2 &\equiv (b[m] = d \wedge b[n] = c) [u / b[n]] \\
&\equiv (b[m] = d) [u / b[n]] \wedge (b[n] = c) [u / b[n]] \\
&\equiv (b[m]) [u / b[n]] = d \wedge (u = c) \\
&\equiv (\text{if } m = n \text{ then } u \text{ else } b[m] \text{ fi}) = d \wedge (u = c) \quad \# \text{ Not a predicate} \\
&\mapsto (\text{if } m = n \text{ then } u = d \text{ else } b[m] = d \text{ fi}) \wedge (u = c) \\
&\mapsto (m = n \wedge u = d \vee m \neq n \wedge b[m] = d) \wedge (u = c)
\end{aligned}$$

$$\begin{aligned}
q_1 &\equiv ((\text{if } m = n \text{ then } u \text{ else } b[m] \text{ fi}) = d \wedge (u = c)) [b[n] / b[m]] \\
&\equiv (\text{if } m = n \text{ then } u \text{ else if } m = m \text{ then } b[n] \text{ else } b[m] \text{ fi fi}) = d \wedge (u = c) \\
&\equiv (\text{if } m = n \text{ then } u \text{ else } b[n] \text{ fi}) = d \wedge (u = c) \quad \# \text{ Not a predicate} \\
&\mapsto (m = n \wedge u = d \vee m \neq n \wedge b[n] = d) \wedge (u = c)
\end{aligned}$$

$$\begin{aligned}
q_0 &\equiv ((\text{if } m = n \text{ then } u \text{ else } b[n] \text{ fi}) = d \wedge (u = c)) [b[m] / u] \\
&\equiv (\text{if } m = n \text{ then } b[m] \text{ else } b[n] \text{ fi}) = d \wedge (b[m] = c) \\
&\quad \# \text{ Let's try to optimize } q_0. \\
&\quad \# m = n \text{ implies } b[m] = b[n] \\
&\mapsto b[n] = d \wedge (b[m] = c)
\end{aligned}$$

It is obvious that the precondition $b[m] = c \wedge b[n] = d \Leftrightarrow q_0$ so the proof is done (and we can remove the condition q_0 from the proof outline).